



Today's agenda

↳ Flatten binary tree to LinkedList

↳ Convert a binary tree to a circular DLL



AlgoPrep

BT

```
class Node {  
    int val;  
    Node left;  
    Node right;  
}
```

3

LL

```
class Node {  
    int val;  
    Node next;  
}
```

3

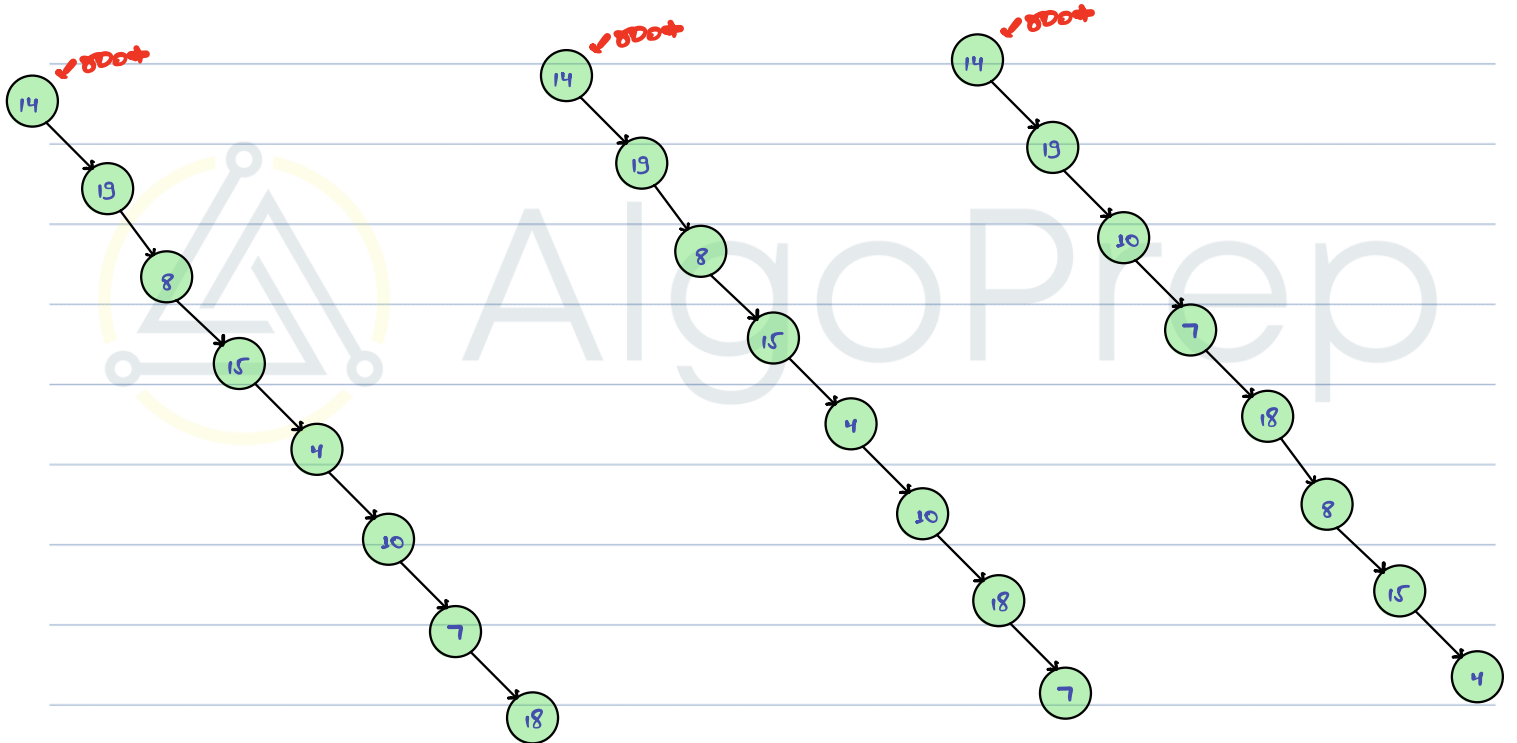
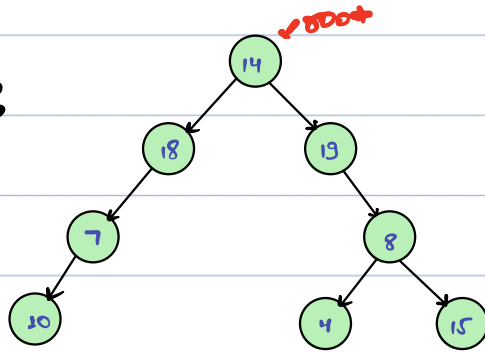


Q) Flatten Binary tree to LinkedList

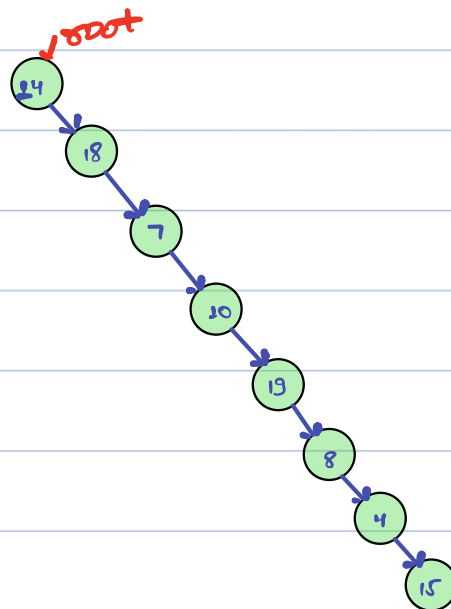
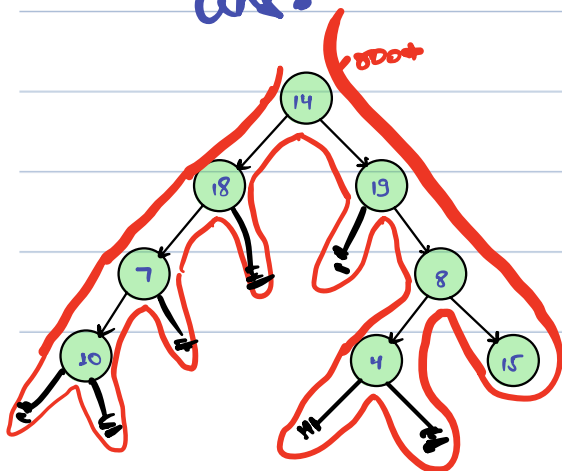
↳ you have to convert Binary tree to LinkedList,
"right" should be considered next and left should be null.

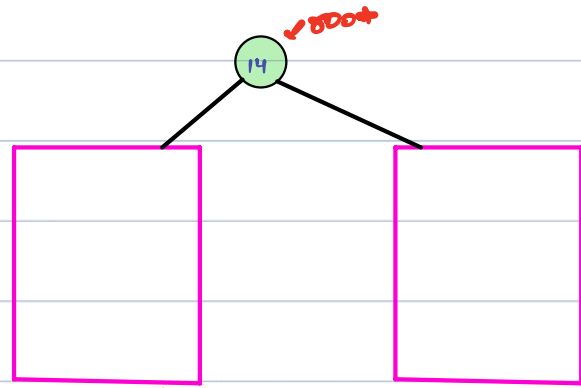
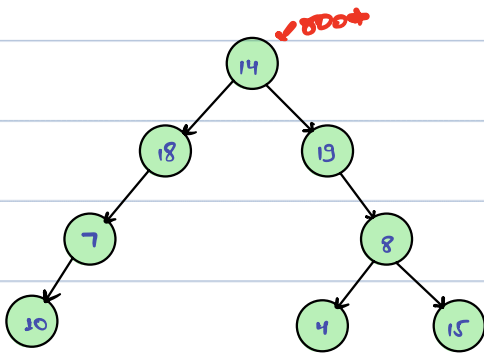
↳ Preorder of tree.

Ex:



Ans:





Null
Null
Non-Null
Non-Null

Null → return root
Non-Null → return root
Null
Non-Null

```
Node flattenmain(Node root){  
    flatten(root);  
    return root;  
}
```

```
Node flatten(Node root){  
    if(root == null){ return null; }
```

```
Node lt = flatten(root.left);
```

```
Node rt = flatten(root.right);
```

```
if(lt == null && rt == null){  
    return root;
```

}

```
else if(lt == null && rt != null){  
    return rt;
```

T.C: O(N)
S.C: O(H)

}

```
else if(lt != null && rt == null){
```

```
Node lc = root.left;
```

```
root.right = lc;
```

```
root.left = null;  
return rt;
```

}

```
else {
```

```
Node lc = root.left;
```

```
Node rc = root.right;
```

```
root.right = lc;
```

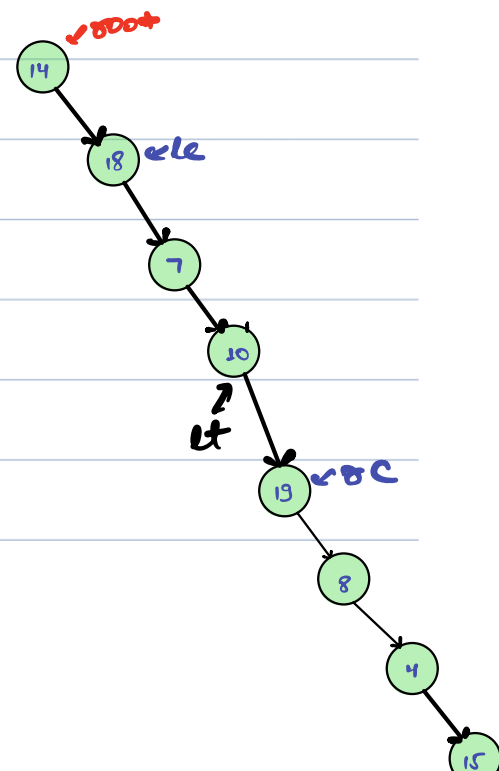
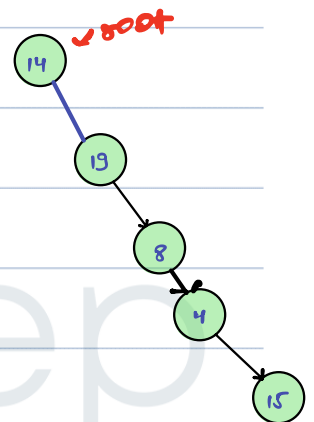
```
root.left = null;
```

```
lt = right = rc;
```

```
return rt;
```

}

}





Node flatten (Node root) {

1 if (root == null) { return null; }

2 Node lt = flatten (root.left);

3 Node rt = flatten (root.right);

if (lt == null && rt == null) {

return root;

}

else if (lt == null && rt != null) {

return rt;

}

4 else if (lt != null && rt == null) {

Node lc = root.left;

root.right = lc;

root.left = null;

return lc;

}

else {

Node lc = root.left;

Node rc = root.right;

root.right = lc;

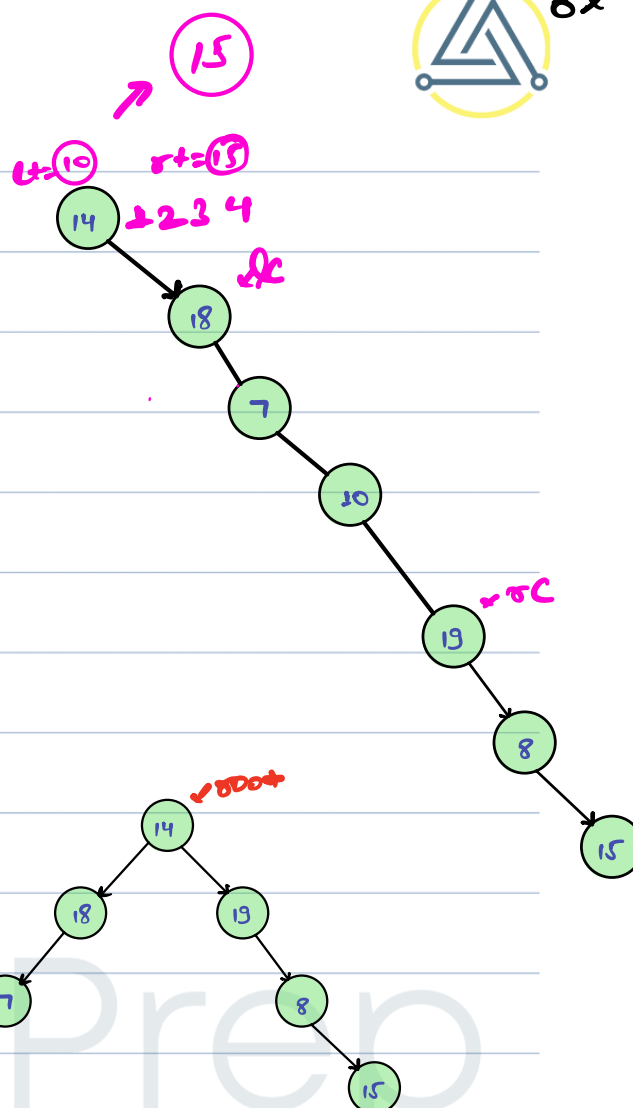
root.left = null;

lc.right = rc;

return rc;

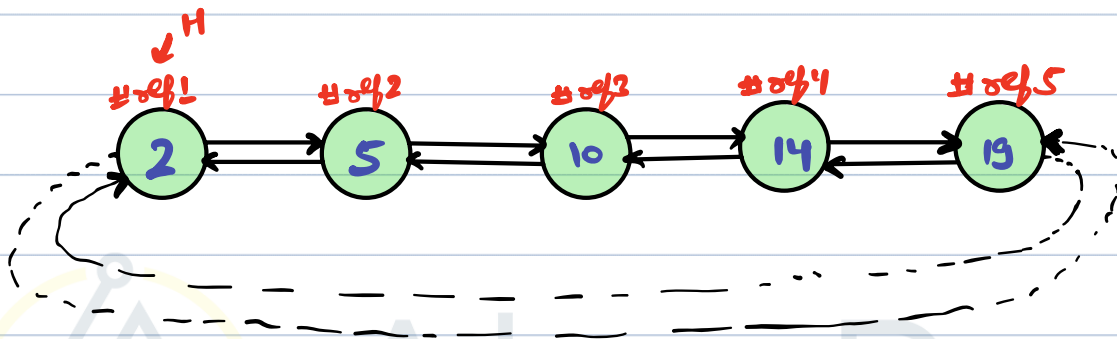
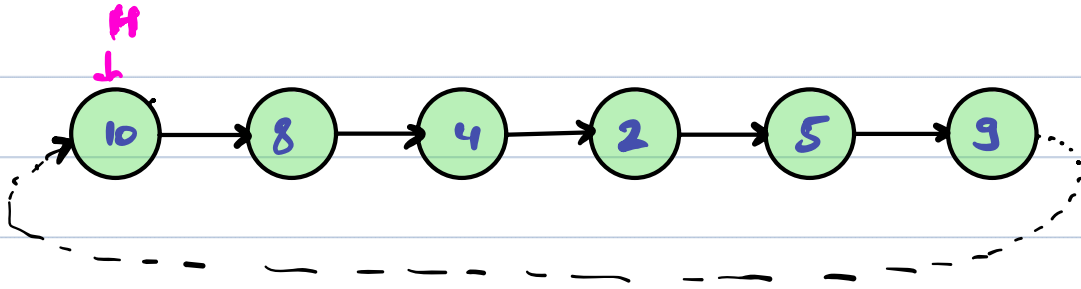
}

}





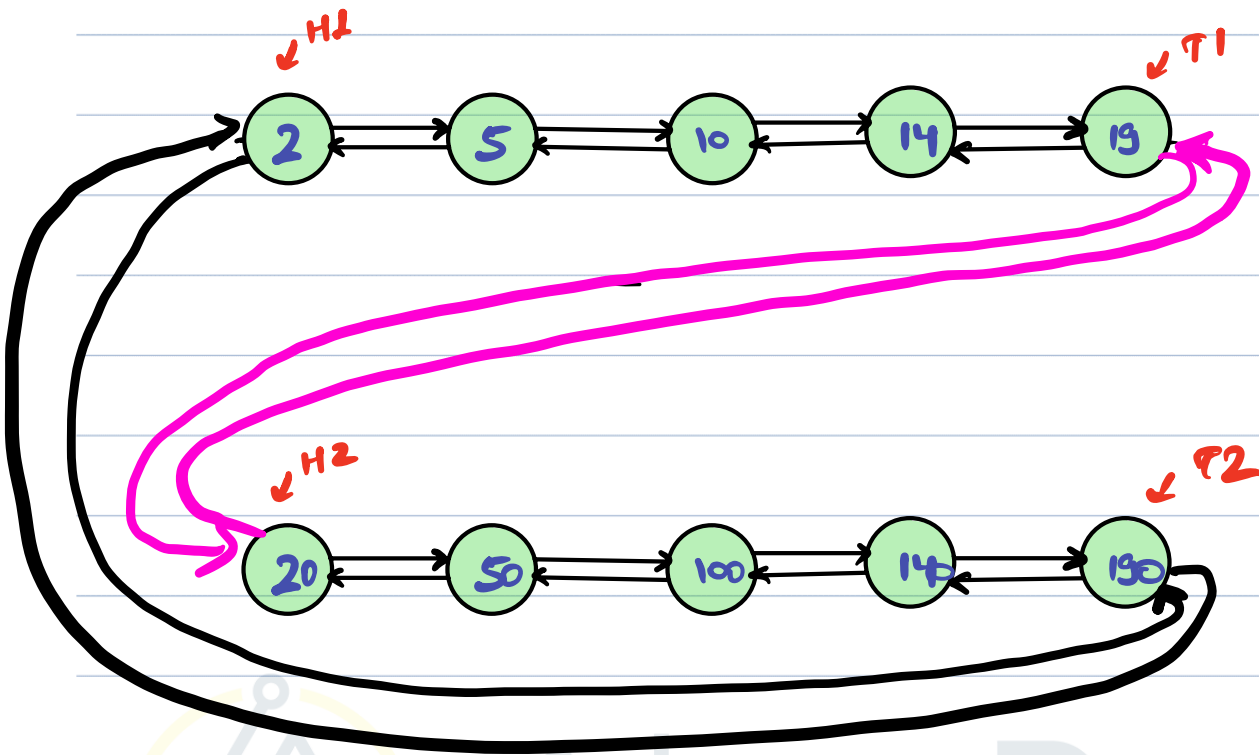
// Circular LinkedList



AlgoPrep



Q) Combine given two circular doubly LinkedList.



Node Concatenate (Node H1, Node H2)

Node T1 = H1. Prev;

Node T2 = H2. Prev;

T1.next = H2;

H2. Prev = T1;

T2.next = H1;

H1. Prev = T2;

return H1;

BT

3

LL

Class Node {

int val;
Node right;
Node left;

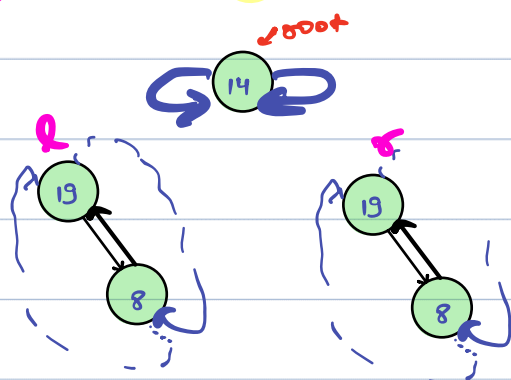
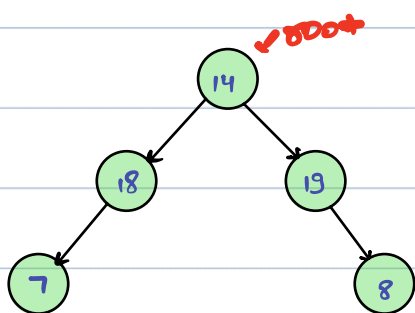
Class Node {

int val;
Node next;
Node prev;



Q) Convert Binary tree to a Circular DLL

↳ follow Inorder. → LNR



Node btreeDLL(Node root) {

if (root == null) { return null; }

Node l = btreeDLL(root.left);

Node r = btreeDLL(root.right);

if (l == null && r == null) {

root.left = root.right = root;

return root;

}

else if (l == null && r != null) {

root.left = root.right = root;

concatenate(root, r);

return root;

}

else if (l != null && r == null) {

root.left = root.right = root;

concatenate(l, root);

return l;

}

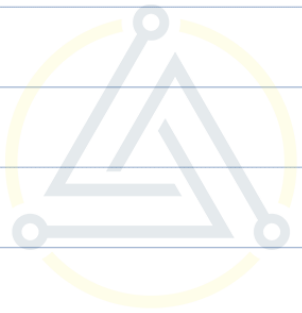
else {

root.left = root.right = root;

3

}

```
Concatenate(l, root);  
Concatenate(l, r);  
return l;
```



AlgoPrep